

Mathématiques appliquées au développement web

Fascicule pédagogique - Bachelor 3

« Le code que vous écrivez tous les jours repose sur les épaules de mathématiciens - et le plus beau, c'est que vous n'êtes pas obligés de devenir mathématiciens pour en profiter. Mais comprendre ce qu'il y a sous le capot, c'est ce qui sépare un développeur exécutant d'un développeur créatif. »

Avant-propos

Ce fascicule n'est PAS un cours de mathématiques. C'est un **cours de culture mathématique pour développeurs web**.

Dans ce document, nous allons survoler des notions mathématiques qui font **tourner le web** : Google, Netflix, Spotify, HTTPS, l'IA dans le navigateur, les animations CSS, les jeux 3D, la blockchain.

À chaque fois, le même schéma : une idée mathématique => du code => un produit que des millions d'utilisateurs utilisent.

L'objectif n'est pas que vous sachiez **calculer** des vecteurs propres ou démontrer un théorème. L'objectif est que vous **reconnaissez** les concepts mathématiques quand vous les croisez dans un projet, que vous **sachiez où chercher** quand vous voulez apprendre, et surtout que vous **développiez l'intuition** que **les maths sont une boîte à outils créative**.

Quand vous fermerez ce fascicule, vous devriez avoir une seule envie : **bidouiller un projet** où une idée mathématique vous sert à fabriquer quelque chose de cool.

PARTIE 1 - HISTOIRE	2
PARTIE 2 - PRÉSENT	5
PARTIE 3 - FUTUR.....	8
PARTIE 4 - EXPLORER.....	10
Résumé en 10 points clés	12

PARTIE 1 - HISTOIRE

Les mathématiques comme moteur des innovations web

Le web n'est pas tombé du ciel. Chaque grande innovation web (chercher, compresser, sécuriser, afficher en 3D...) repose sur **une idée mathématique précise**, souvent vieille de plusieurs siècles, qui a été appliquée à un problème nouveau au bon moment.

1.1 La compression : ranger l'image, le son, la vidéo dans un mouchoir de poche

JPEG : la transformée en cosinus discrète (DCT)

Le problème (1980-1990) : une image brute en RGB, c'est énorme. Une photo 12 MP non compressée pèse environ 36 Mo. Impossible à transmettre sur les modems 56k de l'époque.

L'idée mathématique : au lieu de stocker l'image **pixel par pixel** (domaine spatial), on la stocke comme une **somme d'ondes** (domaine fréquentiel). C'est l'idée fondamentale derrière la **transformée en cosinus discrète** (Discrete Cosine Transform, DCT), inventée en 1974 par Nasir Ahmed, T. Natarajan et K. R. Rao à l'Université du Texas à Arlington.

Pourquoi ça marche : une photo réelle contient surtout des variations **basses fréquences** (un ciel bleu uniforme, un visage lisse). Les **hautes fréquences** (détails fins) représentent peu d'information visuelle. En jetant les hautes fréquences inutiles, on **compresse sans que l'œil voie la différence**.

Comment JPEG l'utilise :

1. L'image est découpée en blocs de 8×8 pixels.
2. Sur chaque bloc, on applique la DCT 2D.
3. Les coefficients sont **quantifiés** (on arrondit grossièrement les hautes fréquences).
4. Le résultat est encodé en Huffman (compression sans perte).

Concept clé - DCT : c'est une cousine de la transformée de Fourier qui exprime un signal comme somme de cosinus. Pour une image, ça revient à dire : « *ton bloc 8×8, c'est tant de patron uniforme + tant de dégradé horizontal + tant de damier...* ». Une image typique a 80% d'énergie concentrée dans 10 coefficients sur 64.

Impact web : le format .jpg représente l'écrasante majorité des images sur le web. Sans DCT, pas d'Instagram, pas de e-commerce, pas de Google Images.

MP3 : l'analyse de Fourier appliquée à l'oreille humaine

Quand vous écoutez un fichier MP3, vous écoutez **le théorème de Fourier appliqué à un modèle psychoacoustique**.

- **Idée 1 (Fourier, 1822) :** tout signal périodique se décompose en somme de sinusoïdes pures. Un son complexe = un assemblage de fréquences pures.
- **Idée 2 (psychoacoustique, années 1980) :** l'oreille humaine n'entend pas tout. Un son fort masque un son faible voisin (**masquage auditif**).
- **MP3 (1993) :** on calcule la transformée (MDCT, une variante de la DCT) du signal audio, on identifie les fréquences que l'oreille ne percevra pas, on les supprime. Résultat : selon Wikipédia, MP3 « peut couramment atteindre une réduction de taille de 75 à 95 % selon le débit » (typiquement un ratio de 10:1 à 14:1 tout en préservant une qualité quasi-CD à l'oreille).

Vidéo H.264, H.265, AV1

Une vidéo, c'est 30 images par seconde. Si on compressait chaque image isolément avec JPEG, ce serait déjà bien. Mais on peut faire mieux : la plupart des images se ressemblent (entre deux frames consécutives, peu de pixels changent).

Idée mathématique : la prédiction par compensation de mouvement. On encode une frame complète (I-frame), puis on encode juste les **différences vectorielles** par rapport à la précédente (P-frame, B-frame). C'est ce qui permet à YouTube et Netflix de streamer en 4K.

Idée de projet : écrire un mini-encodeur d'image en JavaScript qui applique une DCT 8×8 sur un canvas, vous laisse régler un curseur de qualité, et vous montre l'image reconstruite. Vous verrez **visuellement** la magie de la compression.

1.2 La cryptographie : sécuriser le web avec de la théorie des nombres

Avant 1976, chiffrer une communication imposait que les deux interlocuteurs **partagent un secret** (un mot de passe) à l'avance. Sur Internet, où vous voulez acheter chez un marchand que vous n'avez jamais rencontré, c'est impossible.

Diffie-Hellman (1976) : se mettre d'accord sur un secret en public

L'astuce mathématique : deux personnes peuvent se mettre d'accord sur un nombre secret **en s'échangeant uniquement des informations publiques**, en utilisant l'**arithmétique modulaire**.

- L'opération $g^a \bmod p$ (élever g à la puissance a , modulo p) est facile à calculer.
- L'opération inverse - retrouver a connaissant $g^a \bmod p$ - est extrêmement difficile (problème du **logarithme discret**).

C'est ce qu'on appelle une **fonction trappe** : facile dans un sens, ultra-coûteuse dans l'autre.

RSA (1977) : chiffrement à clé publique

Trois mathématiciens du MIT - Ron Rivest, Adi Shamir, Leonard Adleman - proposent un système où **chacun a deux clés** : une **publique** (que tout le monde peut voir) et une **privée** (gardée secrète).

L'astuce : le système repose sur le fait qu'**il est facile de multiplier deux grands nombres premiers, mais extrêmement difficile de factoriser leur produit**. Multiplier deux nombres premiers de 1024 bits prend une milliseconde. Factoriser leur produit (2048 bits) prendrait des milliards d'années aux ordinateurs actuels.

Courbes elliptiques (ECC, années 2000-2010) : plus efficace, plus moderne

Mêmes principes, mais sur un objet géométrique (la courbe elliptique $y^2 = x^3 + ax + b$). Avantage : **clés beaucoup plus courtes** pour la même sécurité. Une clé ECC de 256 bits offre la sécurité d'une clé RSA de 3072 bits.

C'est ce que TLS 1.3 utilise par défaut aujourd'hui (X25519 pour l'échange de clés, ECDSA P-256 ou Ed25519 pour les signatures).

Concept clé - Fonction trappe : une opération mathématique facile dans un sens, infaisable dans l'autre **sauf si on connaît un secret**. Toute la cryptographie moderne repose là-dessus.

Anecdote historique : le concept de cryptographie à clé publique avait en réalité été découvert **secrètement en 1969-1973** par James Ellis, Clifford Cocks et Malcolm Williamson au GCHQ britannique. Mais classifié secret défense, ce travail est resté caché jusqu'en 1997. Diffie-Hellman et RSA ont donc redécouvert publiquement ce qui existait déjà dans des coffres-forts militaires.

1.3 Les moteurs de recherche : Google, ou comment classer le web avec des vecteurs propres

Avant Google (1996) : AltaVista, Lycos, Excite. Ces moteurs cherchaient des mots-clés et renvoyaient les pages où ces mots apparaissaient le plus souvent. Résultat : les spammeurs bourraient leurs pages de mots-clés et trusaient les premiers résultats.

1998 - Brin et Page, doctorants à Stanford : ils ont une intuition géniale.

« Une page est importante si elle est citée (liée) par d'autres pages importantes. »

C'est récursif - et c'est précisément ça qui en fait un magnifique problème mathématique.

L'idée mathématique : on modélise le web comme une **gigantesque matrice de transition** (chaque ligne/colonne = une page web, chaque case = la probabilité de passer d'une page à l'autre via un lien). Le score d'importance de chaque page est le **vecteur propre** dominant de cette matrice (correspondant à la valeur propre 1).

Interprétation intuitive : imaginez un internaute aléatoire qui clique au hasard sur les liens des pages qu'il visite. Au bout d'un long moment, il passe **plus de temps** sur les pages importantes. La probabilité qu'il s'y trouve à un instant donné = le PageRank.

Brin et Page rapportent dans leur article original « The Anatomy of a Large-Scale Hypertextual Web Search Engine » (1998) que sur un graphe de 322 millions de liens, **52 itérations** suffisent pour converger.

Impact : PageRank a permis à Google de surclasser tous les concurrents en quelques mois. Aujourd'hui, le PageRank de 1998 n'est plus qu'un signal parmi des centaines, mais l'idée fondatrice - **classer par importance dérivée d'un graphe** - irrigue toute l'IA moderne (réseaux sociaux, recommandation, PageRank biologique en bio-informatique).

Concept clé - Vecteur propre : un vecteur qui, après transformation par une matrice, **garde sa direction** (seule sa longueur change). Pour le PageRank, c'est l'état stationnaire d'une marche aléatoire sur le graphe du web.

1.4 Graphisme 3D et rendu : matrices, transformations, shaders

Tout ce qui s'affiche en 3D dans votre navigateur - un jeu Three.js, une visite virtuelle, Google Maps - repose sur **trois opérations matricielles fondamentales** :

1. **Translation** (déplacer)
2. **Rotation** (tourner)
3. **Scale** (mettre à l'échelle)

Chaque transformation est représentée par une matrice 4x4. Les composer revient à les multiplier. Pour afficher un objet 3D sur votre écran 2D, le pipeline applique **plusieurs matrices à la suite** :

Coord. objet → [Matrice modèle] → Coord. monde
→ [Matrice vue] → Coord. caméra
→ [Matrice proj.] → Coord. écran

Le shader (programme exécuté pour chaque pixel sur le GPU) calcule la couleur finale en fonction des lumières, textures, normales - c'est de l'algèbre linéaire en parallèle massif.

Une note sur les quaternions : pour représenter les rotations, on utilise rarement des angles d'Euler (problème de **gimbal lock** : deux axes peuvent s'aligner et bloquer une dimension). On utilise des

quaternions, des nombres à 4 dimensions inventés par Hamilton en 1843, parfaitement adaptés aux interpolations fluides entre orientations (SLERP).

1.5 Algorithmes de tri, recherche, optimisation : la complexité

Le problème éternel : comment faire pour qu'un programme reste rapide quand la quantité de données explose ?

La réponse mathématique : la **notation Big O**, qui décrit comment le temps d'exécution évolue avec la taille n de l'entrée.

Complexité	Exemple	$n=1\ 000$
$O(1)$	Accès à un élément de tableau par index	1 op
$O(\log n)$	Recherche dichotomique	~ 10 op
$O(n)$	Parcourir une liste	1 000 op
$O(n \log n)$	Tri merge sort, quicksort	$\sim 10\ 000$ op
$O(n^2)$	Tri à bulles, comparer toutes les paires	1 000 000 op
$O(2^n)$	Brute force sur sous-ensembles	10^{301} op

Implications web :

- Si votre fonction de recherche est en $O(n^2)$, elle marche sur 100 utilisateurs et plante sur 10 000.
- Les structures de données (Map en $O(1)$, Array indexé en $O(1)$, Set en $O(1)$...) ne sont pas un détail de syntaxe : ce sont des choix de complexité.

Concept clé - Big O : une mesure de **comment le coût grandit** avec la taille des données, pas du coût absolu. Un algorithme $O(n \log n)$ bat presque toujours un algorithme $O(n^2)$ à grande échelle, même s'il a une constante plus grande.

PARTIE 2 - PRÉSENT

Où sont les maths dans le web aujourd'hui ?

2.1 Data et A/B testing : prendre des décisions avec des chiffres

Quand Booking.com, Airbnb ou Spotify changent un bouton, ils ne le font pas au feeling. Ils lancent un **A/B test** : 50% des utilisateurs voient la version A, 50% la version B. Au bout d'un certain temps, on compare les taux de conversion.

Le piège : si A convertit à 10,3% et B à 10,5%, est-ce que B est vraiment meilleur ? Ou c'est juste du bruit ?

Les outils mathématiques :

- **L'intervalle de confiance** : « avec 95% de confiance, le vrai taux de conversion est entre 9,3% et 11,3% ».
- **Le test d'hypothèse** : on rejette l'hypothèse nulle « A et B sont équivalents » avec une certaine probabilité d'erreur (la fameuse **p-value** < 0,05).

- **Le théorème central limite** : justifie qu'on peut approximer la distribution des taux par une loi normale dès que l'échantillon est assez grand.

Exemple JS minimal :

```
// Test Z pour comparer deux proportions
function zTest(convA, nA, convB, nB) {
  const pA = convA / nA;
  const pB = convB / nB;
  const pPool = (convA + convB) / (nA + nB);
  const se = Math.sqrt(pPool * (1 - pPool) * (1/nA + 1/nB));
  const z = (pB - pA) / se;
  return { lift: pB - pA, zScore: z, significant: Math.abs(z) > 1.96 };
}
```

2.2 Recommandation : Netflix, YouTube, Spotify

Lorsque Netflix vous suggère un film, il résout un problème mathématique formidable : **prédire les notes que vous donneriez à des films que vous n'avez pas vus.**

Filtrage collaboratif (idée des années 1990) : « les utilisateurs qui aiment ce que tu aimes ont aussi aimé X ».

Factorisation matricielle (Netflix Prize 2006-2009) : on imagine une gigantesque matrice utilisateurs × films pleine de trous (les notes manquantes). On la **décompose** en deux petites matrices : utilisateurs × **facteurs latents** et films × **facteurs latents**.

Les facteurs latents sont des dimensions cachées (« amour de l'action », « préférence pour les films français », « tolérance au gore »...) que l'algorithme **découvre tout seul**. Ces facteurs latents sont les **embeddings** modernes.

Embeddings (état de l'art 2020+) : chaque item (mot, vidéo, produit, utilisateur) est représenté par un **vecteur de quelques centaines de dimensions**. Des items similaires ont des vecteurs proches dans l'espace. La recherche se fait par **plus proches voisins** (cosine similarity).

Concept clé - Embedding : transformer un objet (mot, image, utilisateur) en un vecteur dans un espace de dimension élevée, où la **proximité géométrique = proximité sémantique**. C'est la base de tout l'IA moderne (ChatGPT, recherche sémantique, recommandation).

2.3 Animations et UI : Bézier, easing, interpolation

Quand vous écrivez en CSS :

```
.bouton {
  transition: transform 0.3s cubic-bezier(0.25, 0.1, 0.25, 1);
}
```

Vous utilisez une **courbe de Bézier cubique** - inventée par Pierre Bézier chez Renault en 1962 pour dessiner les carrosseries de voiture.

Comment ça marche : la courbe est définie par deux points fixes (début et fin) et **deux points de contrôle** que vous définissez. La courbe transforme un temps $t \in [0, 1]$ en une progression y (qui peut sortir de $[0, 1]$ pour créer des effets de rebond).

Mot-clé CSS	Équivalent cubic-bezier
linear	cubic-bezier(0, 0, 1, 1)
ease	cubic-bezier(0.25, 0.1, 0.25, 1)
ease-in	cubic-bezier(0.42, 0, 1, 1)
ease-out	cubic-bezier(0, 0, 0.58, 1)
ease-in-out	cubic-bezier(0.42, 0, 0.58, 1)

Au-delà du CSS : Framer Motion, GSAP, Motion One - toutes ces bibliothèques JS d'animation utilisent les mêmes courbes de Bézier. Maîtriser cubic-bezier, c'est passer d'animations « yawn » à des animations qui **respirent**.

2.4 Jeux et simulations dans le navigateur

Un jeu de plateforme dans Canvas, c'est : - Des **vecteurs 2D** (position, vitesse, accélération). - Une boucle d'**intégration numérique** : position += velocity * dt; velocity += acceleration * dt. - De la **détection de collision** (AABB, circle-circle, SAT pour les polygones). - Optionnellement un moteur physique (Matter.js, Cannon.js) qui résout des équations différentielles.

```
// Mini physique 2D avec gravité et rebond
const ball = { x: 0, y: 0, vx: 5, vy: 0 };
const gravity = 0.5, restitution = 0.8;
function update(dt) {
  ball.vy += gravity * dt;
  ball.x += ball.vx * dt;
  ball.y += ball.vy * dt;
  if (ball.y > floor) { ball.y = floor; ball.vy *= -restitution; }
}
```

2.5 Visualisation de données : D3.js

D3 (Data-Driven Documents), créé par Mike Bostock en 2011, repose sur deux abstractions mathématiques :

- **Les échelles (scales)** : des fonctions qui transforment un **domaine de données** en un **range de pixels**. scaleLinear, scaleLog, scaleTime, scaleOrdinal...
- **Les projections géographiques** : projeter une sphère (la Terre) sur un plan (l'écran). D3 supporte Mercator, équirectangulaire, Albers, orthographique... Chaque projection a ses **distorsions** (Mercator étire les pôles, Albers conserve les surfaces).

```
const x = d3.scaleLinear().domain([0, 100]).range([0, 800]);
x(50); // → 400 pixels
```

2.6 Sécurité web : hash, bcrypt, JWT, CSRF

Hashing = transformer une donnée en une empreinte de taille fixe, **non réversible**.

- SHA-256 produit 256 bits. Utilisé pour signer, vérifier l'intégrité.
- bcrypt (1999, Niels Provos & David Mazières) : conçu pour être **lent**. On l'utilise pour stocker les mots de passe : même un attaquant qui vole la base ne pourra pas brute-forcer les mots de passe.

JWT (JSON Web Token) : trois parties séparées par des points : header.payload.signature. La signature, c'est typiquement HMAC-SHA256(header + payload, secret). Le serveur vérifie qu'on n'a pas modifié le token sans connaître la clé.

// Pseudo-code de vérification JWT (HS256)

```
const expected = HMAC_SHA256(header_b64 + "." + payload_b64, SECRET);
if (expected === signature_b64) { /* token valide */ }
```

CSRF tokens : un identifiant aléatoire par session, vérifié sur chaque requête sensible.

Mathématiquement : tirer une chaîne dans un espace si grand qu'elle n'est pas devinable.

2.7 IA dans le navigateur (2025-2026)

C'est sans doute la révolution la plus enthousiasmante du moment. Trois piliers technologiques ont convergé :

- **WebGPU** : API qui donne aux pages web un accès direct au GPU (Chrome 113+ depuis avril 2023, Safari 26 en septembre 2025, Firefox 141 en juillet 2025). Inclut les **compute shaders** indispensables pour l'IA.
- **WebAssembly (WASM) avec SIMD** : exécution natif-like dans le navigateur.
- **Modèles quantifiés** : compresser un LLM de 5 Go à 500 Mo via la quantification 4-bit.

Frameworks à connaître : - **TensorFlow.js** (Google, modèles prêts à l'emploi : reconnaissance d'image, pose detection). - **Transformers.js** (Hugging Face, par Joshua Lochner / Xenova) : pipeline('sentiment-analysis') en une ligne. - **WebLLM** (équipe MLC, CMU/UW) : exécute des LLM (Llama, Phi-2, SmolLM) directement dans Chrome via WebGPU. - **ONNX Runtime Web** (Microsoft) : exécute n'importe quel modèle ONNX dans le navigateur.

```
import { pipeline } from '@huggingface/transformers';
const generator = await pipeline('text-generation', 'Xenova/distilgpt2');
const output = await generator('Hello');
```

Concept clé - Inférence côté client : le modèle d'IA tourne **chez l'utilisateur**, pas sur votre serveur. Avantages : confidentialité (les données ne sortent jamais du navigateur), zéro coût d'API, fonctionne hors ligne. Inconvénient : limite mémoire - les threads de discussion Chromium documentent un plafond de **4 Go par onglet** (Chromium Issue #416284 : « *We limit to 4Gb because certain types of attacks rely on being able to allocate >4Gb of memory* »).

PARTIE 3 - FUTUR

Les mathématiques qui vont transformer le web

3.1 WebGPU et calcul parallèle massif

WebGPU n'est pas seulement « WebGL en mieux ». C'est l'arrivée des **compute shaders** dans le navigateur - ces programmes qui tournent sur des milliers de cœurs GPU en parallèle.

Ce qu'on peut faire avec un compute shader (WGSL) :

- Multiplier des matrices 1024×1024 en quelques millisecondes (utile pour le ML).
- Simuler des fluides, des particules, des automates cellulaires en temps réel.
- Faire du traitement d'image avancé (filtres, edge detection, segmentation) directement sur GPU.

Statut au début 2026 : WebGPU est désormais activé par défaut dans Chrome, Edge, Safari et Firefox (annoncé en novembre 2025). On peut commencer à viser le grand public.

3.2 LLM locaux dans le navigateur

WebLLM et Transformers.js permettent d'exécuter des modèles comme **SmolLM2** (360M et 1.7B paramètres, HuggingFaceTB), **Phi-2** (2.7B, Microsoft) ou **Llama-3.2-1B** (Meta) entièrement côté client. Selon le papier WebLLM (arXiv:2412.15803, équipe MLC/CMU), un **modèle 3B quantifié 4-bit décode 90 tokens/seconde sur un MacBook Apple M3** ; sur M2, les benchmarks publics situent typiquement Llama-3.1 8B autour de 18 tok/s et des petits modèles (Qwen 0.8B) à 55 tok/s via Metal.

Conséquences pour les développeurs web :

- Chatbots offline, privés, sans coût d'API.
- Recherche sémantique locale (vector DB embarquée dans IndexedDB).
- Assistants de code, transcription Whisper en temps réel, captioning d'images, OCR ; et ce sans serveur.

3.3 Simulation temps réel : physique, fluides, particules

Avec les compute shaders, on peut simuler : - Des **fluides** (méthodes SPH, FLIP, ou Navier-Stokes simplifié). - Des **systèmes de particules** de millions d'unités (galaxies, fumée, foules). - Des **soft bodies** (tissus, jellies) avec des contraintes ressort.

Couplé à WebXR (réalité virtuelle/augmentée dans le navigateur), on entre dans l'ère des **expériences spatiales** sans installation.

3.4 Génération procédurale : créer du contenu à l'infini

Bruit de Perlin : Ken Perlin a développé l'algorithme en travaillant chez Mathematical Applications Group (MAGI) sur le film *Tron* de Disney (1982). Il l'a formellement publié au SIGGRAPH 1985 (« An Image Synthesizer ») et a reçu un Academy Award for Technical Achievement en 1997, avec la citation officielle : « *To Ken Perlin for the development of Perlin Noise, a technique used to produce natural appearing textures on computer generated surfaces for motion picture visual effects.* » L'algorithme génère du **hasard lisse** (continu, sans cassures), idéal pour les textures organiques, les terrains, les nuages.

```
// Avec p5.js
function draw() {
  for (let x = 0; x < width; x++) {
    let y = noise(x * 0.01) * height; // Perlin 1D
    point(x, y); // crée une ligne ondulée naturelle
  }
}
```

Automates cellulaires : Game of Life de Conway (1970), génération de cavernes pour roguelikes, simulation de feux de forêt.

L-systems (Aristid Lindenmayer, 1968) : grammaires formelles qui modélisent la croissance végétale. Avec quelques règles de réécriture, on génère un arbre fractal réaliste.

```
// L-system simple
const axiom = 'F';
const rules = { 'F': 'F+F-F-F+F' }; // courbe de Koch
function expand(s, n) {
  for (let i = 0; i < n; i++) {
```

```

s = s.split("").map(c => rules[c] || c).join("");
}
return s;
}

```

3.5 Réalité virtuelle/augmentée (WebXR)

WebXR permet de créer des expériences VR/AR dans le navigateur, accessibles via un Quest, un Vision Pro, un téléphone. Les maths sous-jacentes :

- **Quaternions** pour les rotations (pas de gimbal lock).
- **Matrices 4×4 homogènes** pour combiner translation + rotation + scale.
- **Lerp et SLERP** pour interpoler positions et orientations entre frames.

3.6 Cryptographie avancée : zero-knowledge, homomorphique, post-quantique

Zero-Knowledge Proofs (ZKP)

Une preuve mathématique qui démontre qu'on **connaît un secret** sans le révéler.

Exemple : prouver qu'on a plus de 18 ans **sans révéler sa date de naissance**. Prouver qu'on a fait un calcul **sans révéler les données**.

Variantes : **zk-SNARKs** (Zcash, Mina), **zk-STARKs** (StarkWare), **PLONK**, **Bulletproofs**. En 2025, l'écosystème Web3 affiche selon CoinGecko plus de 11 milliards de dollars de capitalisation autour des projets ZK et 3,5 milliards de volume quotidien.

Homomorphic Encryption (FHE)

Faire des calculs sur des données chiffrées **sans les déchiffrer**. Schémas modernes : **CKKS**, **BGV**, **TFHE**. Use cases 2025 : ML privé (le serveur calcule des prédictions sans voir vos données), médecine, finance.

Cryptographie post-quantique

Si un jour un ordinateur quantique de taille suffisante existe, l'algorithme de Shor cassera RSA et ECC en quelques heures. Le NIST a déjà standardisé en 2024 trois algorithmes post-quantiques : **CRYSTALS-Kyber** (chiffrement), **CRYSTALS-Dilithium** et **SPHINCS+** (signatures). Les navigateurs commencent à intégrer ces algorithmes (Chrome a déployé Kyber dans TLS dès 2023).

Idée de projet : créer une démo de ZK proof simple en JavaScript (via la lib `snarkjs` ou `circom`) - par exemple, prouver qu'on connaît la solution d'un Sudoku sans révéler la solution.

PARTIE 4 - EXPLORER

Quelles mathématiques apprendre pour aller plus loin ?

Cette partie est un **menu de pistes**. Vous n'avez pas besoin de tout maîtriser. Choisissez **un domaine** qui vous attire et plongez.

4.1 Probabilités et statistiques

Pourquoi c'est utile :

- A/B testing rigoureux.
- Comprendre les modèles ML (toute la régression, classification est probabiliste).
- Recommandation, détection d'anomalies.

- Estimer la taille d'échantillon nécessaire.

Quelques exemples concrets : Calculer la **bonne durée d'un test A/B** avant lancement ; Coder un système de notation **bayésien** (« 5 étoiles avec 1 vote » vs « 4,8 avec 1000 votes ») ; Détecter du spam avec un classifieur naïf bayésien.

4.2 Algèbre linéaire

Pourquoi c'est utile :

- Le langage de tout l'IA moderne (un réseau de neurones = une suite de multiplications matricielles).
- WebGL, Three.js, transformations 3D.
- PageRank, recommandation, embeddings. - Compression (PCA, SVD).

Quelques exemples concrets : Implémenter un **transformateur 2D** (rotation + scale + translation) avec des matrices 3×3 ; Faire de la **réduction dimensionnelle** sur un jeu de données pour le visualiser ; Coder une recherche par cosine similarity sur des embeddings.

4.3 Géométrie

Pourquoi c'est utile :

- Canvas, SVG, animations 2D.
- WebGL, jeux 3D.
- Détection de collisions.
- Layout (CSS Grid, Flexbox sont des problèmes de pavage).

Quelques exemples concrets : Détecter si un point est dans un polygone ; Implémenter un algorithme de **Voronoi** pour des effets visuels (D3 le fournit) ; Créer une animation de courbes de Bézier interactives.

4.4 Théorie des graphes

Pourquoi c'est utile :

- Réseaux sociaux (qui suit qui).
- Routing (Dijkstra, A* dans les jeux et la cartographie).
- Dépendances (npm, webpack, build systems).
- Le DOM est un arbre, donc un graphe.

Exemples concrets : Implémenter Dijkstra pour trouver le plus court chemin sur une carte ; Détecter les **cycles** dans un graphe de dépendances ; Visualiser un réseau social avec un layout force-directed (D3).

4.5 Optimisation

Pourquoi c'est utile :

- Performance des algorithmes.
- Entraînement des modèles ML (descente de gradient).
- Layout, scheduling, allocation de ressources.

Exemples concrets : Implémenter une régression linéaire **avec gradient descent** en JavaScript ; Optimiser un layout (problème NP-difficile pour la version générale) ; Tuner les hyperparamètres d'un modèle.

4.6 Algorithmique et complexité

Pourquoi c'est utile :

- Tout. Vraiment, tout. Choisir une structure de données = choisir une complexité.
- Les entretiens techniques.

Exemples concrets : Mémoization, programmation dynamique (LeetCode) ; Choix entre Array, Map, Set, Object en JS selon les besoins ; Implémenter votre propre cache LRU.

Résumé en 10 points clés

1. **Les maths ne sont pas dans le code, elles SONT le code.** Un `Array.sort()` cache un algorithme dont la complexité dépend du nombre d'éléments. Une transition CSS cache une courbe de Bézier. Un cadenas HTTPS cache RSA ou ECC.
2. **PageRank = algèbre linéaire appliquée à un graphe.** L'idée que « important = cité par des choses importantes » est mathématiquement un vecteur propre. Cette même idée fonde aujourd'hui les réseaux sociaux, la recommandation et l'IA.
3. **La compression repose sur l'analyse fréquentielle.** JPEG, MP3, MP4 partagent la même intuition : décomposer un signal en briques plus simples (cosinus, sinus), garder l'essentiel, jeter le reste.
4. **La cryptographie web repose sur des fonctions trappes.** Faciles dans un sens, infaisables dans l'autre - sauf si on connaît un secret. C'est ce qui rend possible HTTPS, JWT, signatures numériques.
5. **Les embeddings sont la lingua franca de l'IA moderne.** Tout objet (mot, image, utilisateur, son) peut devenir un vecteur, et tout problème peut alors se résoudre par proximité dans l'espace.
6. **Les courbes de Bézier rendent vos animations vivantes.** Comprendre `cubic-bezier()` transforme votre UX. Les keywords par défaut (`ease`, `ease-in`) sont un point de départ, pas une finalité.
7. **L'IA dans le navigateur est arrivée.** En 2025-2026, vous pouvez exécuter des LLM, de la transcription audio, du captioning d'images **côté client**, sans serveur, sans clé API.
8. **WebGPU change la donne.** Les compute shaders dans le navigateur, désormais activés par défaut dans tous les navigateurs majeurs depuis novembre 2025, permettent du calcul scientifique et de l'IA à très haute performance.
9. **La complexité algorithmique compte plus que la micro-optimisation.** Un $O(n \log n)$ bat un $O(n^2)$ à grande échelle. Mieux vaut choisir la bonne structure de données qu'optimiser une boucle.
10. **Vous n'avez pas besoin d'être mathématicien pour utiliser les maths.** Vous avez besoin de **reconnaître les concepts** quand vous les croisez, et de **savoir où chercher**. Le reste, c'est de la curiosité.